01 Numpy

Plan for this lecture:

1. Numpy Introduction
2. Contrast Memory allocation of Array and Python List
3. Matrix Multiplication and Vectorisation
4. Exercises

Introduction to Numpy

- Numerical Python (NumPy) is a package full of methods that can perform useful operations on data.
- NumPy provides a convenient API (Application Programmable Interface) that provides a way to 'interface' with / operate on data.
- It reintroduces types which is more coding but more efficient way to search/sort/store data than the 'loosely' typed nature of Python that we've seen so far.
- More documentation available at: https://numpy.org

Numpy Arrays vs Python List

- NumPy arrays are different to Python Lists.
- NumPy arrays reintroduce the 'typed' nature of more 'verbose' languages (C, C++, Java), where everything is explicitly typed.
- NumPy arrays operate like arrays from C and Java where they declared to store data of one type (only integers), unlike Python and JS, which can store data of different types.
- NumPy arrays therefore data is 'cast' – floating point numbers to integers, or in some cases – an error is produced (strings to integers).

Getting started with Numpy

- You'll either need to install this if you're in VSC.
- OR if you're in Anaconda or Google Colab, you should have access to Numpy already... just need to import it.

```
!pip install numpy
```

```
pip install numpy
```

```
python3 -m pip install -U numpy --user
```

In [1…
```
import time
import timeit
import sys

import numpy as np
np
```
Out[1…
```
<module 'numpy' from '/Users/nick/Library/Python/3.9/lib/python/site-packages/numpy/__init__.py'>
```

In [1…
```
l = [1,2,3,4,5,6]
type(l)
```
Out[2…
```
list
```

In [2…
```
a = np.array([1,2,3,4,5,6])
a
```
Out[2…
```
array([1, 2, 3, 4, 5, 6])
```

In [2…
```
type(a)
```
Out[2…
```
numpy.ndarray
```

Type of data

- NumPy Arrays are homogeneous, meaning all elements are of the same data type. This uniformity allows for more efficient storage and computation.
- Python Lists are heterogeneous, capable of storing elements of different data types, which adds overhead to manage type information.

Heterogeneous vs Homogeneous data

- `numpy.array` stores homogeneous data (same type) - for memory allocation
- Python's `list` can store hetrogeneous data (different types) – as point to objects - flexible size.

Notice below how one floating point number will upcast all the integers to floats

In [3…
```
l = [1,2.0, True, "Nick"]
l
```
Out[3…
```
[1, 2.0, True, 'Nick']
```

In [3…
```
type(l[3])
```
Out[3…
```
str
```

In [2…
```
a = np.array([3.14,2,3,4,5])
a
```
Out[2…
```
array([3.14, 2.  , 3.  , 4.  , 5.  ])
```

In [3…
```
a = np.array([1,2.0, True, "Nick"])
a
```
Out[3…
```
array(['1', '2.0', 'True', 'Nick'], dtype='<U32')
```

In [4…
```
type(a[3])
```
Out[4…
```
numpy.str_
```

In […
```
a = np.array([1,2,3],dtype='float32')
a
```
```
array([1., 2., 3.], dtype=float32)
```

In [1…
```
a[0].dtype
```
Out[1…
```
dtype('float32')
```

In [1…
```
a = np.array([1,2,3],dtype='float16')
a
```
Out[1…
```
array([1., 2., 3.], dtype=float16)
```

In [1…
```
a[0].dtype
```
Out[1…
```
dtype('float16')
```

Python's memory overhead

- Python objects have significant internal overhead due to reference counting, type information, and dynamic memory allocation.
- `Integer` (28 bytes): The integer 42 is stored as a **Python object**, including the overhead for reference counting and type information, even though the actual value only requires a few bytes.
- `List` (80 bytes): The `list` object itself has overhead, and Python typically over-allocates memory to allow for efficient growth. The actual data stored in the `list` (the integer objects 1, 2, and 3) are not included in the size reported by `sys.getsizeof()`. Each of those elements has its own overhead.
- `String` (54 bytes): Like `lists`, `strs` in Python are objects with associated metadata, even though the actual `str` content is small.

In [4…
```
import sys
```

In [4…
```
a = np.array([1,2,3],dtype='float32')
a
```
Out[4…
```
array([1., 2., 3.], dtype=float32)
```

In [4…
```
a.nbytes
```
Out[4…
```
12
```

In [4…
```
i = 0
for x in a:
    print(x, ":", a[i].nbytes)
    i += 1
```
```
1.0 : 4
2.0 : 4
3.0 : 4
```

In [4…
```
sys.getsizeof(a)
```
Out[4…
```
124
```

In [6…
```
i = 0
for x in a:
    print(x, ":", sys.getsizeof(a[i]))
    i += 1
```
```
1.0 : 28
2.0 : 28
3.0 : 28
```

In [7…
```
py_list = [0, 1.0, "N", True]
py_list
```
Out[7…
```
[0, 1.0, 'N', True]
```

In [7…
```
py_list[0] = 50
```

In [7…
```
py_list[0].bit_length()
```
Out[7…
```
6
```

In [7…
```
sys.getsizeof(py_list)
```
Out[7…
```
120
```

In [7…
```
i = 0
for x in py_list:
    print(x, ":", sys.getsizeof(py_list[i]))
    i += 1
```
```
50 : 28
1.0 : 24
N : 50
True : 28
```

In [1…
```
tuple_ex = tuple(range(1000))
list_ex = list(range(1000))
numpy_ex = np.array([range(1000)])
print("Space taken by tuple =",tuple_ex.__sizeof__()," bytes")
print("Space taken by list =",list_ex.__sizeof__()," bytes")
print("Space taken by NumPy array =",numpy_ex.__sizeof__()," bytes")
```
```
Space taken by tuple = 8024  bytes
Space taken by list = 8040  bytes
Space taken by NumPy array = 8128  bytes
```

In [1…
```
import numpy as np
import sys

# a. Create structures
size = 1000000
py_list = list(range(size))
np_array = np.arange(size)

# b. Check memory usage
print("Memory size of Python list:", sys.getsizeof(py_list), "bytes")
print("Memory size of NumPy array:", np_array.nbytes, "bytes")
```
```
Memory size of Python list: 8000056 bytes
Memory size of NumPy array: 8000000 bytes
```

In [1…
```
type(py_list[10])
```
Out[1…
```
int
```

In [1…
```
py_list[10].bit_length()
```
Out[1…
```
4
```

In [1…
```
sys.getsizeof(py_list[10])
```
Out[1…
```
28
```

In [1…
```
type(np_array[10])
```
Out[1…
```
numpy.int64
```

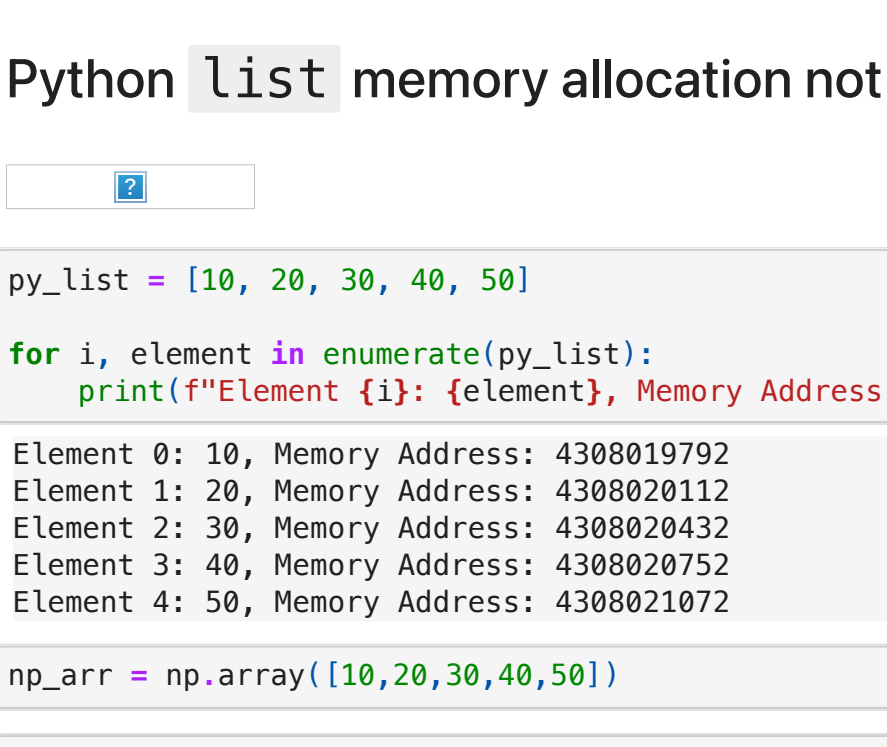
In [1…
```
np_array[10].nbytes
```
Out[1…
```
8
```

In [1…
```
sys.getsizeof(np_array[10])
```
Out[1…
```
32
```

`numpy.array` Contiguous memory allocation

- NumPy Arrays store data in contiguous blocks of memory, which enhances cache performance and allows for more efficient access and manipulation.
- Python Lists, on the other hand, store references to objects scattered throughout memory, leading to slower access times, especially for large datasets.
- NumPy arrays are generally faster and more efficient for numerical operations compared to Python lists, primarily due to their contiguous memory layout and vectorized operations implemented in optimized C code.

Question: How many bytes are these elements apart?
Question: What is the bit size of the data type?



Python `list` memory allocation not contiguous

In [4…
```
py_list = [10, 20, 30, 40, 50]

for i, element in enumerate(py_list):
    print(f"Element {i}: {element}, Memory Address: {id(element)}")
```
```
Element 0: 10, Memory Address: 4308019792
Element 1: 20, Memory Address: 4308020112
Element 2: 30, Memory Address: 4308020432
Element 3: 40, Memory Address: 4308020752
Element 4: 50, Memory Address: 4308021072
```

In [5…
```
np_arr = np.array([10,20,30,40,50])
```

In [5…
```
base_address = np_arr.__array_interface__['data'][0]

print("NumPy array memory address:", base_address)
```
```
NumPy array memory address: 5773674192
```

In [5…
```
np_arr.dtype
```
Out[5…
```
dtype('int64')
```

In [5…
```
element_size = np_arr.itemsize
print(element_size)
```
```
8
```

- 64 bits = 8 bytes

In [5…
```
for i in range(len(np_arr)):
    element_address = base_address + i * element_size
    print(f"Element {i}: {np_arr[i]}, Memory Address: {element_address}")
```
```
Element 0: 10, Memory Address: 5773674192
Element 1: 20, Memory Address: 5773674200
Element 2: 30, Memory Address: 5773674208
Element 3: 40, Memory Address: 5773674216
Element 4: 50, Memory Address: 5773674224
```

- 64 bit = 8 bytes
- Therefore the addresses are 8 bytes apart.

2-Dimensional Arrays

- Need an inner and outer pair of `[ ]` brackets to generate rows and columns.
- Remember that element access for a Python `list` is `[row][col]` - NOT `[row,col]`
- `numpy.arrays` can use either `[,]` or `[][]` but may need paretheses to in certain contexts where operation is ambigious.

In [5…
```
a = np.array([[1,2,3],[4,5,6]])
a
```
Out[5…
```
array([[1, 2, 3],
       [4, 5, 6]])
```

In [6…
```
a[1][1]
```
Out[6…
```
5
```

In [6…
```
a[1,1]
```
Out[6…
```
5
```

In [6…
```
py_2D = [[1,2,3], [4,5,6]]
py_2D
```
Out[6…
```
[[1, 2, 3], [4, 5, 6]]
```

In [9…
```
py_2D[1][1]
```
Out[9…
```
5
```

In [9…
```
py_2D[1,1]
```
```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In [92], line 1
----> 1 py_2D[1,1]

TypeError: list indices must be integers or slices, not tuple
```

`np` array functions

In [6…
```
a = np.arange(10)
a
```
Out[6…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [1…
```
py_list = list(range(0,10))
py_list
```
Out[1…
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [6…
```
a = np.arange(19,50,5)
a
```
Out[6…
```
array([19, 24, 29, 34, 39, 44, 49])
```

In [2…
```
for i in range(0,10):
    print(i)
```
```
0
1
2
3
4
5
6
7
8
9
```

In [3…
```
for i in py_list:
    print(i)
```
```
0
1
2
3
4
5
6
7
8
9
```

In [9…
```
np.zeros(10)
```
Out[9…
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

In [9…
```
np.ones(10)
```
Out[9…
```
array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

In [7…
```
t = (3,2)
type(t)
```
Out[7…
```
tuple
```

In [9…
```
a = np.ones((3, 2))
a
```
Out[9…
```
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
```

In [7…
```
a = np.array([[1,2,3], [4,5,6]])
a
```
Out[7…
```
array([[1, 2, 3],
       [4, 5, 6]])
```

In [7…
```
a.reshape((3,2))
```
Out[7…
```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

In [7…
```
a = np.full((2, 2), 5)
a
```
Out[7…
```
array([[5, 5],
       [5, 5]])
```

In [7…
```
a = np.full((3, 3), 7)
a
```
Out[7…
```
array([[7, 7, 7],
       [7, 7, 7],
       [7, 7, 7]])
```

In [2…
```
np.eye(3)
```
Out[2…
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

Random generation via `numpy.random`

In [1…
```
a = np.random.random((2,2))
a
```
Out[1…
```
array([[0.59759749, 0.57090874],
       [0.3824511 , 0.02262516]])
```

In [1…
```
a = np.random.randint((2,2))
a
```
Out[1…
```
array([1, 1])
```

In [1…
```
a = np.random.randint(2, size=10)
a
```
Out[1…
```
array([1, 1, 1, 0, 0, 1, 1, 1, 0, 1])
```

In [2…
```
a = np.random.randint(2, size=(2,2))
a
```
Out[2…
```
array([[0, 0],
       [1, 1]])
```

In [1…
```
a = np.random.randint(2, size=(2,3))
a
```
Out[1…
```
array([[1, 0, 0],
       [1, 0, 0]])
```

In [1…
```
a = np.random.randint(9, size=(3,3))
a
```
Out[1…
```
array([[8, 4, 6],
       [2, 0, 6],
       [4, 1, 0]])
```

3D arrays!

In [9…
```
a = np.random.randint(9, size=(3,3,3))
a
```
Out[9…
```
array([[[5, 4, 6],
        [6, 6, 6],
        [4, 8, 4]],

       [[2, 6, 8],
        [4, 4, 6],
        [7, 8, 0]],

       [[0, 8, 5],
        [4, 7, 3],
        [2, 7, 1]]])
```

In [8…
```
a = np.linspace(0, 1, 11)
a
```
Out[8…
```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ])
```

Array attributes

- You've already seen: `nbytes` and `dtype` for memory allocation.
- There are many helpful attributes that describe characteristics of the array for you.

In [1…
```
a = np.random.randint(9, size=(3,3))
print("size:", a.size)
print("shape:", a.shape)
print("dimensions:", a.ndim)
```
```
size: 9
shape: (3, 3)
dimensions: 2
```

In [1…
```
a = np.random.randint(9, size=(3,3,3))
print("size:", a.size)
print("shape:", a.shape)
print("dimensions:", a.ndim)
```
```
size: 27
shape: (3, 3, 3)
dimensions: 3
```

In [2…
```
dir(np)
```
Out[2…
```
['False_',
 'ScalarType',
 'True_',
 '_CopyMode',
 '_NoValue',
 '__NUMPY_SETUP__',
 '__all__',
 '__array_api_version__',
 '__builtins__',
 '__cached__',
 '__config__',
 '__dir__',
 '__doc__',
 '__expired_attributes__',
 '__file__',
 '__former_attrs__',
 '__future_scalars__',
 '__getattr__',
 '__loader__',
 '__name__',
 '__numpy_submodules__',
 '__package__',
 '__path__',
 '__spec__',
 '__version__',
 '_core',
 '_distributor_init',
 '_expired_attrs_2_0',
 '_get_promotion_state',
 '_globals',
 '_int_extended_msg',
 '_mat',
 '_msg',
 '_no_nep50_warning',
 '_pyinstaller_hooks_dir',
 '_pytesttester',
 '_set_promotion_state',
 '_specific_msg',
 '_type_info',
 '_typing',
 '_utils',
 'abs',
 'absolute',
 'acos',
 'acosh',
 'add',
 'all',
 'allclose',
 'amax',
 'amin',
 'angle',
 'any',
 'append',
 'apply_along_axis',
 'apply_over_axes',
 'arange',
 'arccos',
 'arccosh',
 'arcsin',
 'arcsinh',
 'arctan',
 'arctan2',
 'arctanh',
 'argmax',
 'argmin',
 'argpartition',
 'argsort',
 'argwhere',
 'around',
 'array',
 'array2string',
 'array_equal',
 'array_equiv',
 'array_repr',
 'array_split',
 'array_str',
 'asanyarray',
 'asarray',
 'asarray_chkfinite',
 'ascontiguousarray',
 'asfortranarray',
 'asin',
 'asinh',
 'asmatrix',
 'astype',
 'atan',
 'atan2',
 'atanh',
 'atleast_1d',
 'atleast_2d',
 'atleast_3d',
 'average',
 'bartlett',
 'base_repr',
 'binary_repr',
 'bincount',
 'bitwise_and',
 'bitwise_count',
 'bitwise_invert',
 'bitwise_left_shift',
 'bitwise_not',
 'bitwise_or',
 'bitwise_right_shift',
 'bitwise_xor',
 'blackman',
 'block',
 'bmat',
 'bool',
 'bool_',
 'broadcast',
 'broadcast_arrays',
 'broadcast_shapes',
 'broadcast_to',
 'busday_count',
 'busday_offset',
 'busdaycalendar',
 'byte',
 'bytes_',
 'c_',
 'can_cast',
 'cbrt',
 'cdouble',
 'ceil',
 'char',
 'character',
 'choose',
 'clip',
 'clongdouble',
 'column_stack',
 'common_type',
 'complex128',
 'complex64',
 'complexfloating',
 'compress',
 'concat',
 'concatenate',
 'conj',
 'conjugate',
 'convolve',
 'copy',
 'copysign',
 'copyto',
 'core',
 'corrcoef',
 'correlate',
 'cos',
 'cosh',
 'count_nonzero',
 'cov',
 'cross',
 'csingle',
 'ctypeslib',
 'cumprod',
 'cumsum',
 'datetime64',
 'datetime_as_string',
 'datetime_data',
 'deg2rad',
 'degrees',
 'delete',
 'diag',
 'diag_indices',
 'diag_indices_from',
 'diagflat',
 'diagonal',
 'diff',
 'digitize',
 'divide',
 'divmod',
 'dot',
 'double',
 'dsplit',
 'dstack',
 'dtype',
 'dtypes',
 'e',
 'ediff1d',
 'einsum',
 'einsum_path',
 'emath',
 'empty',
 'empty_like',
 'equal',
 'errstate',
 'euler_gamma',
 'exceptions',
 'exp',
 'exp2',
 'expand_dims',
 'expm1',
 'extract',
 'eye',
 'f2py',
 'fabs',
 'fft',
 'fill_diagonal',
 'finfo',
 'fix',
 'flatiter',
 'flatnonzero',
 'flexible',
 'flip',
 'fliplr',
 'flipud',
 'float16',
 'float32',
 'float64',
 'float_power',
 'floating',
 'floor',
 'floor_divide',
 'fmax',
 'fmin',
 'fmod',
 'format_float_positional',
 'format_float_scientific',
 'frexp',
 'from_dlpack',
 'frombuffer',
 'fromfile',
 'fromfunction',
 'fromiter',
 'frompyfunc',
 'fromregex',
 'fromstring',
 'full',
 'full_like',
 'gcd',
 'generic',
 'genfromtxt',
 'geomspace',
 'get_include',
 'get_printoptions',
 'getbufsize',
 'geterr',
 'geterrcall',
 'gradient',
 'greater',
 'greater_equal',
 'half',
 'hamming',
 'hanning',
 'heaviside',
 'histogram',
 'histogram2d',
 'histogram_bin_edges',
 'histogramdd',
 'hsplit',
 'hstack',
 'hypot',
 'i0',
 'identity',
 'iinfo',
 'imag',
 'in1d',
 'index_exp',
 'indices',
 'inexact',
 'inf',
 'info',
 'inner',
 'insert',
 'int16',
 'int32',
 'int64',
 'int8',
 'int_',
 'intc',
 'integer',
 'interp',
 'intersect1d',
 'intp',
 'invert',
 'is_busday',
 'isclose',
 'iscomplex',
 'iscomplexobj',
 'isdtype',
 'isfinite',
 'isfortran',
 'isin',
 'isinf',
 'isnan',
 'isnat',
 'isneginf',
 'isposinf',
 'isreal',
 'isrealobj',
 'isscalar',
 'issubdtype',
 'iterable',
 'ix_',
 'kaiser',
 'kron',
 'lcm',
 'ldexp',
 'left_shift',
 'less',
 'less_equal',
 'lexsort',
 'lib',
 'linalg',
 'linspace',
 'little_endian',
 'load',
 'loadtxt',
 'log',
 'log10',
 'log1p',
 'log2',
 'logaddexp',
 'logaddexp2',
 'logical_and',
 'logical_not',
 'logical_or',
 'logical_xor',
 'logspace',
 'long',
 'longdouble',
 'longlong',
 'ma',
 'mask_indices',
 'matmul',
 'matrix',
 'matrix_transpose',
 'max',
 'maximum',
 'may_share_memory',
 'mean',
 'median',
 'memmap',
 'meshgrid',
 'mgrid',
 'min',
 'min_scalar_type',
 'minimum',
 'mintypecode',
 'mod',
 'modf',
 'moveaxis',
 'multiply',
 'nan',
 'nan_to_num',
 'nanargmax',
 'nanargmin',
 'nancumprod',
 'nancumsum',
 'nanmax',
 'nanmean',
 'nanmedian',
 'nanmin',
 'nanpercentile',
 'nanprod',
 'nanquantile',
 'nanstd',
 'nansum',
 'nanvar',
 'ndarray',
 'ndenumerate',
 'ndim',
 'ndindex',
 'nditer',
 'negative',
 'nested_iters',
 'newaxis',
 'nextafter',
 'nonzero',
 'not_equal',
 'number',
 'object_',
 'ogrid',
 'ones',
 'ones_like',
 'outer',
 'packbits',
 'pad',
 'partition',
 'percentile',
 'permute_dims',
 'pi',
 'piecewise',
 'place',
 'poly',
 'poly1d',
 'polyadd',
 'polyder',
 'polydiv',
 'polyfit',
 'polyint',
 'polymul',
 'polynomial',
 'polysub',
 'polyval',
 'positive',
 'pow',
 'power',
 'printoptions',
 'prod',
 'promote_types',
 'ptp',
 'put',
 'put_along_axis',
 'putmask',
 'quantile',
 'r_',
 'rad2deg',
 'radians',
 'random',
 'ravel',
 'ravel_multi_index',
 'real',
 'real_if_close',
 'rec',
 'recarray',
 'reciprocal',
 'record',
 'remainder',
 'repeat',
 'require',
 'reshape',
 'resize',
 'result_type',
 'right_shift',
 'rint',
 'roll',
 'rollaxis',
 'roots',
 'rot90',
 'round',
 'row_stack',
 's_',
 'save',
 'savetxt',
 'savez',
 'savez_compressed',
 'sctypeDict',
 'searchsorted',
 'select',
 'set_printoptions',
 'setbufsize',
 'setdiff1d',
 'seterr',
 'seterrcall',
 'setxor1d',
 'shape',
 'shares_memory',
 'short',
 'show_config',
 'show_runtime',
 'sign',
 'signbit',
 'signedinteger',
 'sin',
 'sinc',
 'single',
 'sinh',
 'size',
 'sort',
 'sort_complex',
 'spacing',
 'split',
 'sqrt',
 'square',
 'squeeze',
 'stack',
 'std',
 'str_',
 'strings',
 'subtract',
 'sum',
 'swapaxes',
 'take',
 'take_along_axis',
 'tan',
 'tanh',
 'tensordot',
 'test',
 'testing',
 'tile',
 'timedelta64',
 'trace',
 'transpose',
 'trapezoid',
 'trapz',
 'tri',
 'tril',
 'tril_indices',
 'tril_indices_from',
 'trim_zeros',
 'triu',
 'triu_indices',
 'triu_indices_from',
 'true_divide',
 'trunc',
 'typecodes',
 'typename',
 'typing',
 'ubyte',
 'ufunc',
 'uint',
 'uint16',
 'uint32',
 'uint64',
 'uint8',
 'uintc',
 'uintp',
 'ulong',
 'ulonglong',
 'union1d',
 'unique',
 'unique_all',
 'unique_counts',
 'unique_inverse',
 'unique_values',
 'unpackbits',
 'unravel_index',
 'unsignedinteger',
 'unwrap',
 'ushort',
 'vander',
 'var',
 'vdot',
 'vecdot',
 'vectorize',
 'void',
 'vsplit',
 'vstack',
 'where',
 'zeros',
 'zeros_like']
```

Array slicing `[ : ]`

- `array_name[ start : stop : interval]`

```
In [2…  a = np.arange(10)
        a
```

```
Out[2…  array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [1…  print(a[3])
```

```
3
```

```
In [1…  a[:3]
```

```
Out[1…  array([0, 1, 2])
```

```
In [1…  a[3:]
```

```
Out[1…  array([3, 4, 5, 6, 7, 8, 9])
```

```
In [1…  a[3:6]
```

```
Out[1…  array([3, 4, 5])
```

Steps/intervals of 2:

```
In [2…  a
```

```
Out[2…  array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [1…  a[3:6:2]
```

```
Out[1…  array([3, 5])
```

Mutiples of 3

```
In [3…  a[::3]
```

```
Out[3…  array([0, 3, 6, 9])
```

Row and Col access

```
In [8…  a = np.random.randint(9, size=(3,3))
        a
```

```
Out[8…  array([[5, 7, 4],
               [6, 4, 7],
               [7, 3, 2]])
```

All of the first column (column 0) - displayed in a one-dimensional form

```
In [8…  a[:,0]
```

```
Out[8…  array([5, 6, 7])
```

All of the first row (row 0)

```
In [8…  a[0,:]
```

```
Out[8…  array([5, 7, 4])
```

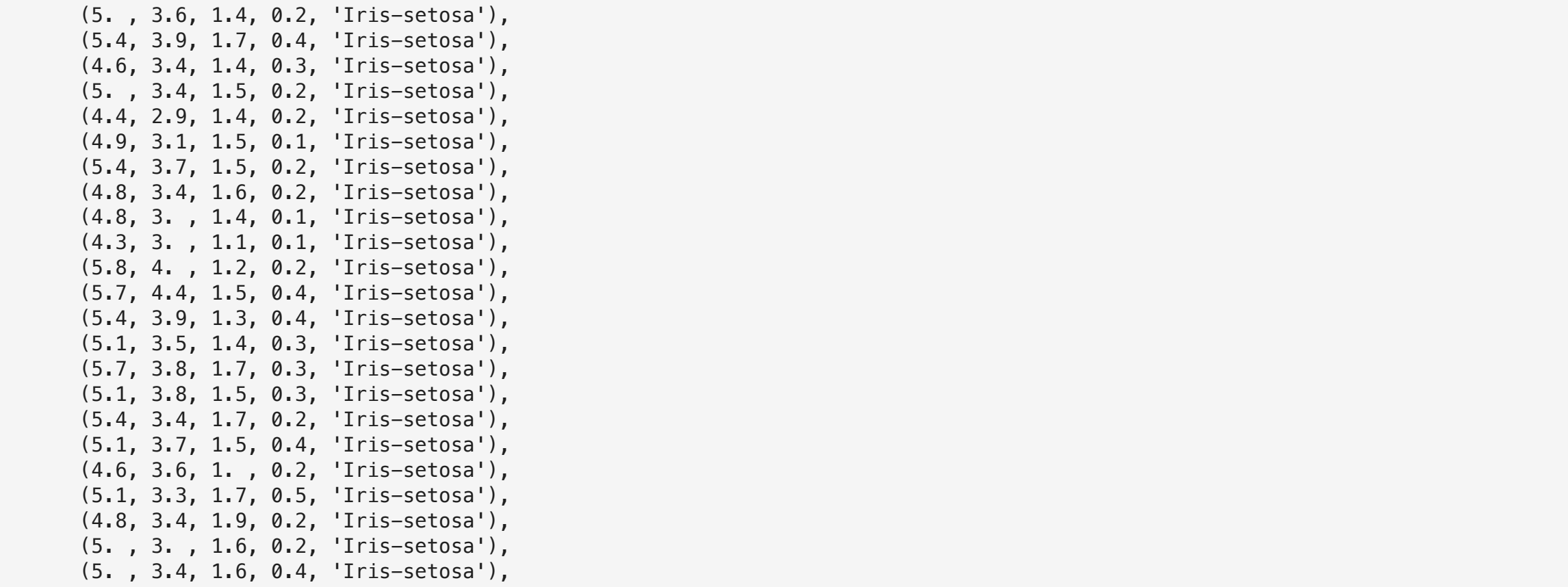
All of the second row (row 1)

```
In [8…  a[1,:]
```

```
Out[8…  array([6, 4, 7])
```

Importing / Reading from files (CSVs)

https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data



```
In [8…  url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'
        iris = np.genfromtxt(url, delimiter=',',
        names=['sepal length', 'sepal width', 'petal length' , 'petal width', 'species'],
        dtype=[np. float64, np.float64, np.float64, np.float64, 'U15']
        )
        iris
```

```
Out[8…  array([(5.1, 3.5, 1.4, 0.2, 'Iris-setosa'),
               (4.9, 3. , 1.4, 0.2, 'Iris-setosa'),
               (4.7, 3.2, 1.3, 0.2, 'Iris-setosa'),
               (4.6, 3.1, 1.5, 0.2, 'Iris-setosa'),
               (5. , 3.6, 1.4, 0.2, 'Iris-setosa'),
               (5.4, 3.9, 1.7, 0.4, 'Iris-setosa'),
               (4.6, 3.4, 1.4, 0.3, 'Iris-setosa'),
               (5. , 3.4, 1.5, 0.2, 'Iris-setosa'),
               (4.4, 2.9, 1.4, 0.2, 'Iris-setosa'),
               (4.9, 3.1, 1.5, 0.1, 'Iris-setosa'),
               (5.4, 3.7, 1.5, 0.2, 'Iris-setosa'),
               (4.8, 3.4, 1.6, 0.2, 'Iris-setosa'),
               (4.8, 3. , 1.4, 0.1, 'Iris-setosa'),
               (4.3, 3. , 1.1, 0.1, 'Iris-setosa'),
               (5.8, 4. , 1.2, 0.2, 'Iris-setosa'),
               (5.7, 4.4, 1.5, 0.4, 'Iris-setosa'),
               (5.4, 3.9, 1.3, 0.4, 'Iris-setosa'),
               (5.1, 3.5, 1.4, 0.3, 'Iris-setosa'),
               (5.7, 3.8, 1.7, 0.3, 'Iris-setosa'),
               (5.1, 3.8, 1.5, 0.3, 'Iris-setosa'),
               (5.4, 3.4, 1.7, 0.2, 'Iris-setosa'),
               (5.1, 3.7, 1.5, 0.4, 'Iris-setosa'),
               (4.6, 3.6, 1. , 0.2, 'Iris-setosa'),
               (5.1, 3.3, 1.7, 0.5, 'Iris-setosa'),
               (4.8, 3.4, 1.9, 0.2, 'Iris-setosa'),
               (5. , 3. , 1.6, 0.2, 'Iris-setosa'),
               (5. , 3.4, 1.6, 0.4, 'Iris-setosa'),
               (5.2, 3.5, 1.5, 0.2, 'Iris-setosa'),
               (5.2, 3.4, 1.4, 0.2, 'Iris-setosa'),
               (4.7, 3.2, 1.6, 0.2, 'Iris-setosa'),
               (4.8, 3.1, 1.6, 0.2, 'Iris-setosa'),
               (5.4, 3.4, 1.5, 0.4, 'Iris-setosa'),
               (5.2, 4.1, 1.5, 0.1, 'Iris-setosa'),
               (5.5, 4.2, 1.4, 0.2, 'Iris-setosa'),
               (4.9, 3.1, 1.5, 0.1, 'Iris-setosa'),
               (5. , 3.2, 1.2, 0.2, 'Iris-setosa'),
               (5.5, 3.5, 1.3, 0.2, 'Iris-setosa'),
               (4.9, 3.1, 1.5, 0.1, 'Iris-setosa'),
               (4.4, 3. , 1.3, 0.2, 'Iris-setosa'),
               (5.1, 3.4, 1.5, 0.2, 'Iris-setosa'),
               (5. , 3.5, 1.3, 0.3, 'Iris-setosa'),
               (4.5, 2.3, 1.3, 0.3, 'Iris-setosa'),
               (4.4, 3.2, 1.3, 0.2, 'Iris-setosa'),
               (5. , 3.5, 1.6, 0.6, 'Iris-setosa'),
               (5.1, 3.8, 1.9, 0.4, 'Iris-setosa'),
               (4.8, 3. , 1.4, 0.3, 'Iris-setosa'),
               (5.1, 3.8, 1.6, 0.2, 'Iris-setosa'),
               (4.6, 3.2, 1.4, 0.2, 'Iris-setosa'),
               (5.3, 3.7, 1.5, 0.2, 'Iris-setosa'),
               (5. , 3.3, 1.4, 0.2, 'Iris-setosa'),
               (7. , 3.2, 4.7, 1.4, 'Iris-versicolor'),
               (6.4, 3.2, 4.5, 1.5, 'Iris-versicolor'),
               (6.9, 3.1, 4.9, 1.5, 'Iris-versicolor'),
               (5.5, 2.3, 4. , 1.3, 'Iris-versicolor'),
               (6.5, 2.8, 4.6, 1.5, 'Iris-versicolor'),
               (5.7, 2.8, 4.5, 1.3, 'Iris-versicolor'),
               (6.3, 3.3, 4.7, 1.6, 'Iris-versicolor'),
               (4.9, 2.4, 3.3, 1. , 'Iris-versicolor'),
               (6.6, 2.9, 4.6, 1.3, 'Iris-versicolor'),
               (5.2, 2.7, 3.9, 1.4, 'Iris-versicolor'),
               (5. , 2. , 3.5, 1. , 'Iris-versicolor'),
               (5.9, 3. , 4.2, 1.5, 'Iris-versicolor'),
               (6. , 2.2, 4. , 1. , 'Iris-versicolor'),
               (6.1, 2.9, 4.7, 1.4, 'Iris-versicolor'),
               (5.6, 2.9, 3.6, 1.3, 'Iris-versicolor'),
               (6.7, 3.1, 4.4, 1.4, 'Iris-versicolor'),
               (5.6, 3. , 4.5, 1.5, 'Iris-versicolor'),
               (5.8, 2.7, 4.1, 1. , 'Iris-versicolor'),
               (6.2, 2.2, 4.5, 1.5, 'Iris-versicolor'),
               (5.6, 2.5, 3.9, 1.1, 'Iris-versicolor'),
               (5.9, 3.2, 4.8, 1.8, 'Iris-versicolor'),
               (6.1, 2.8, 4. , 1.3, 'Iris-versicolor'),
               (6.3, 2.5, 4.9, 1.5, 'Iris-versicolor'),
               (6.1, 2.8, 4.7, 1.2, 'Iris-versicolor'),
               (6.4, 2.9, 4.3, 1.3, 'Iris-versicolor'),
               (6.6, 3. , 4.4, 1.4, 'Iris-versicolor'),
               (6.8, 2.8, 4.8, 1.4, 'Iris-versicolor'),
               (6.7, 3. , 5. , 1.7, 'Iris-versicolor'),
               (6. , 2.9, 4.5, 1.5, 'Iris-versicolor'),
               (5.7, 2.6, 3.5, 1. , 'Iris-versicolor'),
               (5.5, 2.4, 3.8, 1.1, 'Iris-versicolor'),
               (5.5, 2.4, 3.7, 1. , 'Iris-versicolor'),
               (5.8, 2.7, 3.9, 1.2, 'Iris-versicolor'),
               (6. , 2.7, 5.1, 1.6, 'Iris-versicolor'),
               (5.4, 3. , 4.5, 1.5, 'Iris-versicolor'),
               (6. , 3.4, 4.5, 1.6, 'Iris-versicolor'),
               (6.7, 3.1, 4.7, 1.5, 'Iris-versicolor'),
               (6.3, 2.3, 4.4, 1.3, 'Iris-versicolor'),
               (5.6, 3. , 4.1, 1.3, 'Iris-versicolor'),
               (5.5, 2.5, 4. , 1.3, 'Iris-versicolor'),
               (5.5, 2.6, 4.4, 1.2, 'Iris-versicolor'),
               (6.1, 3. , 4.6, 1.4, 'Iris-versicolor'),
               (5.8, 2.6, 4. , 1.2, 'Iris-versicolor'),
               (5. , 2.3, 3.3, 1. , 'Iris-versicolor'),
               (5.6, 2.7, 4.2, 1.3, 'Iris-versicolor'),
               (5.7, 3. , 4.2, 1.2, 'Iris-versicolor'),
               (5.7, 2.9, 4.2, 1.3, 'Iris-versicolor'),
               (6.2, 2.9, 4.3, 1.3, 'Iris-versicolor'),
               (5.1, 2.5, 3. , 1.1, 'Iris-versicolor'),
               (5.7, 2.8, 4.1, 1.3, 'Iris-versicolor'),
               (6.3, 3.3, 6. , 2.5, 'Iris-virginica'),
               (5.8, 2.7, 5.1, 1.9, 'Iris-virginica'),
               (7.1, 3. , 5.9, 2.1, 'Iris-virginica'),
               (6.3, 2.9, 5.6, 1.8, 'Iris-virginica'),
               (6.5, 3. , 5.8, 2.2, 'Iris-virginica'),
               (7.6, 3. , 6.6, 2.1, 'Iris-virginica'),
               (4.9, 2.5, 4.5, 1.7, 'Iris-virginica'),
               (7.3, 2.9, 6.3, 1.8, 'Iris-virginica'),
               (6.7, 2.5, 5.8, 1.8, 'Iris-virginica'),
               (7.2, 3.6, 6.1, 2.5, 'Iris-virginica'),
               (6.5, 3.2, 5.1, 2. , 'Iris-virginica'),
               (6.4, 2.7, 5.3, 1.9, 'Iris-virginica'),
               (6.8, 3. , 5.5, 2.1, 'Iris-virginica'),
               (5.7, 2.5, 5. , 2. , 'Iris-virginica'),
               (5.8, 2.8, 5.1, 2.4, 'Iris-virginica'),
               (6.4, 3.2, 5.3, 2.3, 'Iris-virginica'),
               (6.5, 3. , 5.5, 1.8, 'Iris-virginica'),
               (7.7, 3.8, 6.7, 2.2, 'Iris-virginica'),
               (7.7, 2.6, 6.9, 2.3, 'Iris-virginica'),
               (6. , 2.2, 5. , 1.5, 'Iris-virginica'),
               (6.9, 3.2, 5.7, 2.3, 'Iris-virginica'),
               (5.6, 2.8, 4.9, 2. , 'Iris-virginica'),
               (7.7, 2.8, 6.7, 2. , 'Iris-virginica'),
               (6.3, 2.7, 4.9, 1.8, 'Iris-virginica'),
               (6.7, 3.3, 5.7, 2.1, 'Iris-virginica'),
               (7.2, 3.2, 6. , 1.8, 'Iris-virginica'),
               (6.2, 2.8, 4.8, 1.8, 'Iris-virginica'),
               (6.1, 3. , 4.9, 1.8, 'Iris-virginica'),
               (6.4, 2.8, 5.6, 2.1, 'Iris-virginica'),
               (7.2, 3. , 5.8, 1.6, 'Iris-virginica'),
               (7.4, 2.8, 6.1, 1.9, 'Iris-virginica'),
               (7.9, 3.8, 6.4, 2. , 'Iris-virginica'),
               (6.4, 2.8, 5.6, 2.2, 'Iris-virginica'),
               (6.3, 2.8, 5.1, 1.5, 'Iris-virginica'),
               (6.1, 2.6, 5.6, 1.4, 'Iris-virginica'),
               (7.7, 3. , 6.1, 2.3, 'Iris-virginica'),
               (6.3, 3.4, 5.6, 2.4, 'Iris-virginica'),
               (6.4, 3.1, 5.5, 1.8, 'Iris-virginica'),
               (6. , 3. , 4.8, 1.8, 'Iris-virginica'),
               (6.9, 3.1, 5.4, 2.1, 'Iris-virginica'),
               (6.7, 3.1, 5.6, 2.4, 'Iris-virginica'),
               (6.9, 3.1, 5.1, 2.3, 'Iris-virginica'),
               (5.8, 2.7, 5.1, 1.9, 'Iris-virginica'),
               (6.8, 3.2, 5.9, 2.3, 'Iris-virginica'),
               (6.7, 3.3, 5.7, 2.5, 'Iris-virginica'),
               (6.7, 3. , 5.2, 2.3, 'Iris-virginica'),
               (6.3, 2.5, 5. , 1.9, 'Iris-virginica'),
               (6.5, 3. , 5.2, 2. , 'Iris-virginica'),
               (6.2, 3.4, 5.4, 2.3, 'Iris-virginica'),
               (5.9, 3. , 5.1, 1.8, 'Iris-virginica')],
              dtype=[('sepal_length', '<f8'), ('sepal_width', '<f8'), ('petal_length', '<f8'), ('petal_width', '<f8'), ('species', '<U15')])
```

```
In [8…  iris[0]
```

```
Out[8…  (5.1, 3.5, 1.4, 0.2, 'Iris-setosa')
```

```
In [8…  iris[0][0]
```

```
Out[8…  5.1
```

Sneak preview of Pandas!

- More on Pandas in our next lecture...

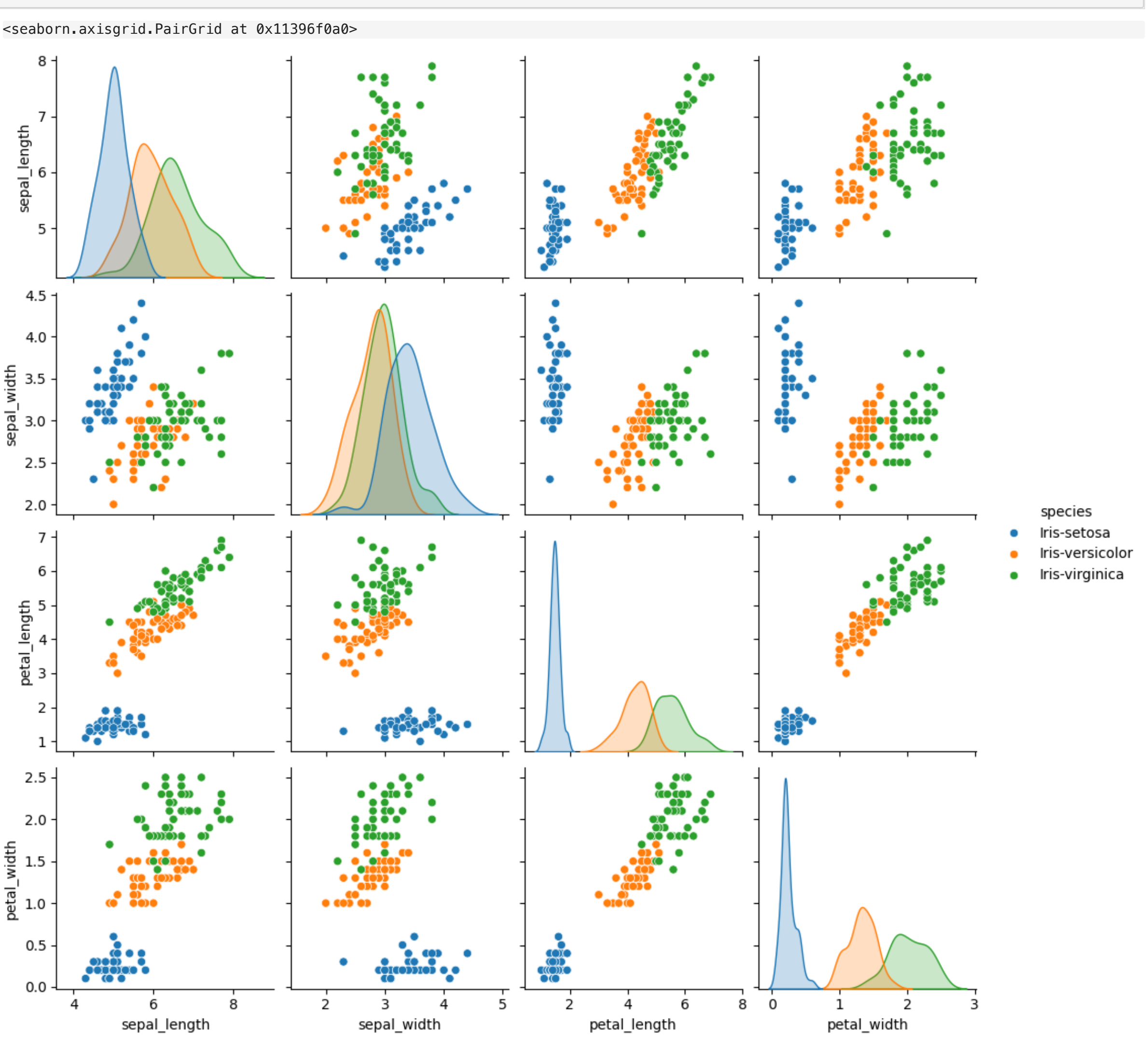
```
In [1…  import pandas as pd
        iris_df = pd.DataFrame(iris)

        iris_df.head()
```

Out[1…

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [1…  import seaborn as sns

        sns.pairplot(iris_df, hue="species")
```

```
Out[1…  <seaborn.axisgrid.PairGrid at 0x11396f0a0>
```



Iteration vs Vectorisation

- Vectorisation is a mathmatical transformation from a matrix to a column vector.

- `numpy` arrays can leverage this ability, allowing operations to be performed on entire arrays without explicit Python loops, and are instead implemented in optimised C and Fortran code 'behind the scenes'.
- Essentially, this allows for parallel processing, rather than sequential / iterative processing.
- Python `lists` require explicit loops or list comprehensions for element-wise operations, which are executed in the slower Python interpreter.

In [1…
```
import numpy as np
import time
import timeit
```

Scenario: Multiply a and b element-wise

```
1 * 4 = 4
2 * 5 = 10
3 * 6 = 18
      +
  sum = 32
```

In [1…
```
a = np.array([1,2,3])
b = np.array([4,5,6])
```

Vectorised operation on `numpy arrays`

In [1…
```
c = np.sum(a * b)
print(c)
```
```
32
```

In [1…
```
print(np.matmul(a,b))
```
```
32
```

Attempting vectorisation on python `lists` - you'll see that the below won't work, as `lists` are not vectorised.

In [4…
```
a = [1,2,3]
b = [4,5,6]

c = sum(a * b)

print(c)
```
```
TypeError                                 Traceback (most recent call last)
Cell In [40], line 4
      1 a = [1,2,3]
      2 b = [4,5,6]
----> 4 c = sum(a * b)
      6 print(c)

TypeError: can't multiply sequence by non-int of type 'list'
```

Therefore, for lists, we have to revert to a non-vectorised (iterative) approach

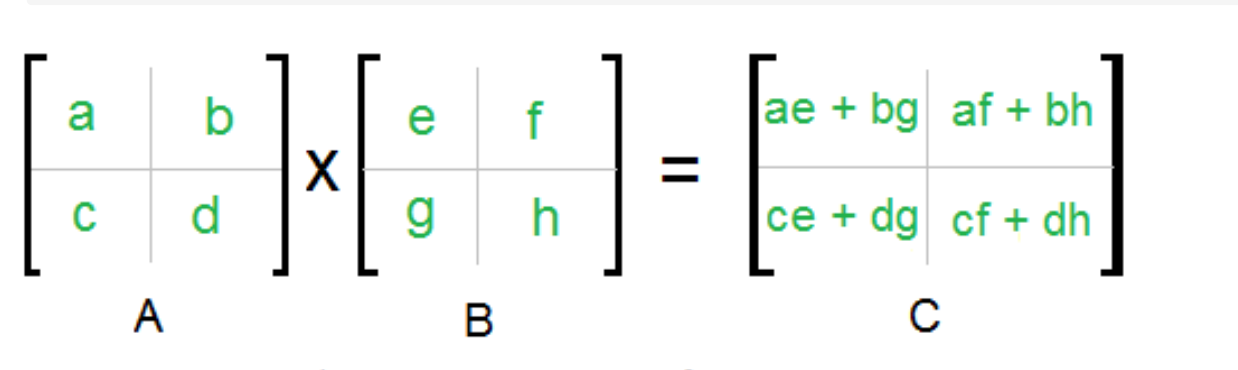
In [4…
```
a = [1,2,3]
b = [4,5,6]

c = 0
for i in range(len(a)):
    c += a[i] * b[i]
print(c)
```
```
32
```

Matrix Multiplication

In [1…
```
a = np.array([[1,2], [3,4]])
b = np.array([[5,6], [7,8]])
```

In [1…
```
a
```
Out[1…
```
array([[1, 2],
       [3, 4]])
```

In [1…
```
b
```
Out[1…
```
array([[5, 6],
       [7, 8]])
```



A, B and C are square metrices of size N x N
a, b, c and d are submatrices of A, of size N/2 x N/2
e, f, g and h are submatrices of B, of size N/2 x N/2

In [1…
```
np.matmul(a,b)
```
Out[1…
```
array([[19, 22],
       [43, 50]])
```

In [4…
```
a @ b
```
Out[4…
```
array([[19, 22],
       [43, 50]])
```

In [1…
```
c = np.sum(a * b)
print(c)
```
```
70
```

Python 2D list, by contrast...

In [1…
```
a_l = [[1,2,3], [3,4,5], [6,7,8]]
b_l = [[5,6,7], [7,8,9], [4,5,6]]
```

In [1…
```
a_l
```
Out[1…
```
[[1, 2, 3], [3, 4, 5], [6, 7, 8]]
```

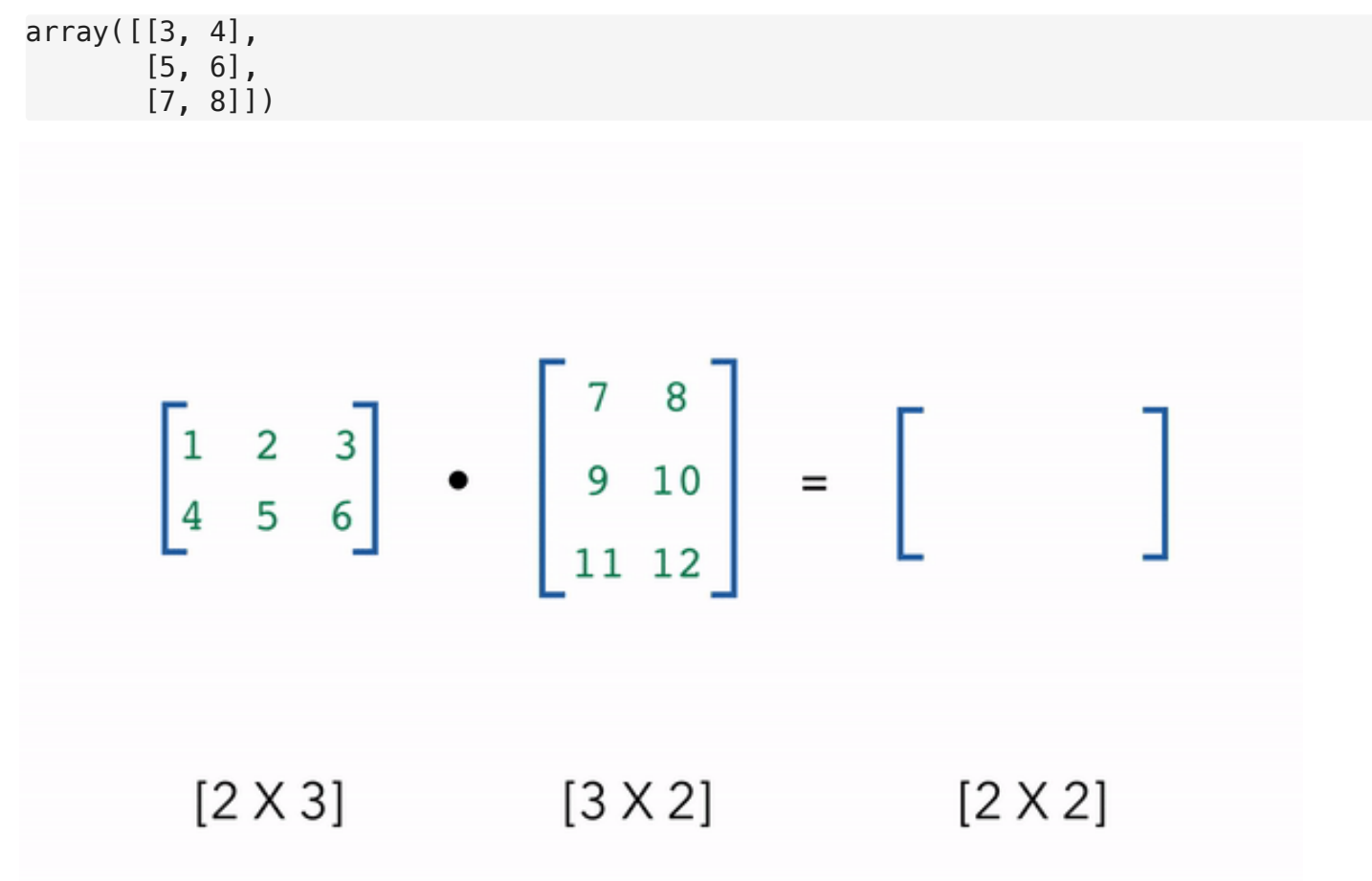
In [1…
```
b_l
```
Out[1…
```
[[5, 6, 7], [7, 8, 9], [4, 5, 6]]
```

In [1…
```
c_l = a_l #for shape
for x in range(len(a_l)):
    for y in range(len(b_l)):
        c[x][y] = a_l[x][y] * b_l[x][y]
c_l
```
Out[1…
```
[[1, 2, 3], [3, 4, 5], [6, 7, 8]]
```

It's also easier to multiply unequally sized 2D arrays with Numpy:

In [1…
```
a = np.array([[1,2,3], [4,5,6]])
a
```
Out[1…
```
array([[1, 2, 3],
       [4, 5, 6]])
```

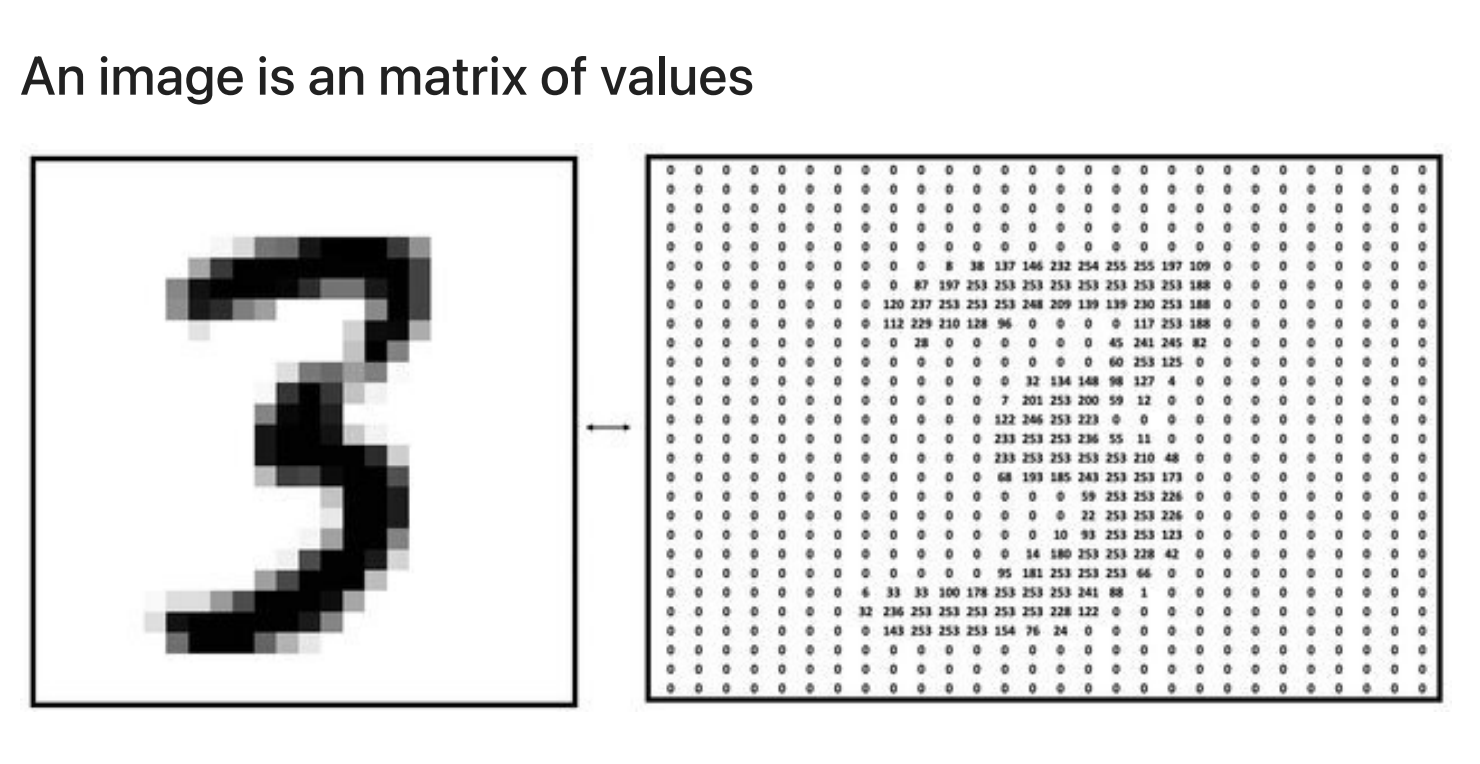
In [1…
```
b = np.array([[3,4], [5,6], [7,8]])
b
```
Out[1…
```
array([[3, 4],
       [5, 6],
       [7, 8]])
```



In [1…
```
np.dot(a,b)
```
Out[1…
```
array([[34, 40],
       [79, 94]])
```

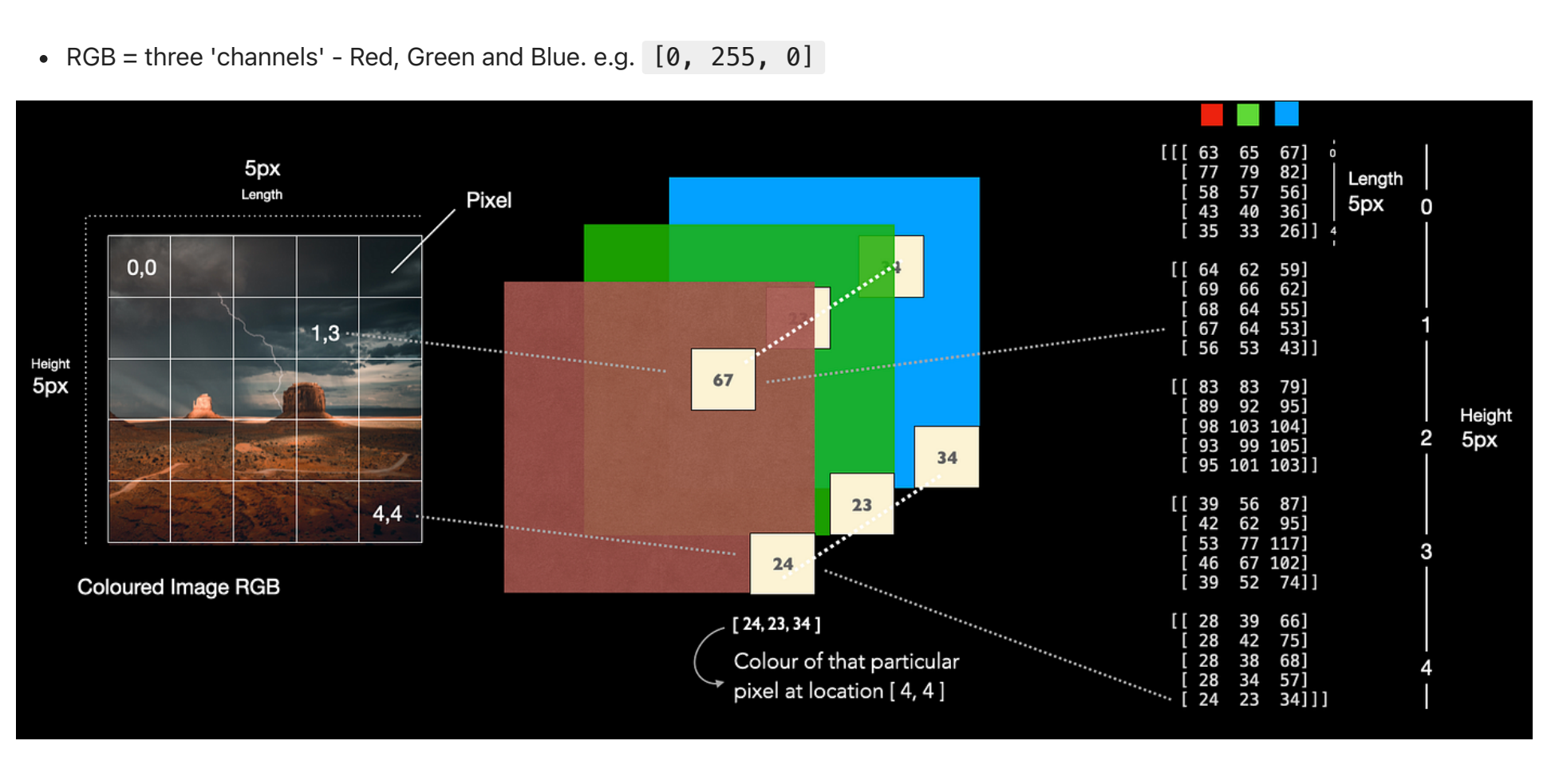
In [1…
```
np.matmul(a,b)
```
Out[1…
```
array([[34, 40],
       [79, 94]])
```

An image is an matrix of values



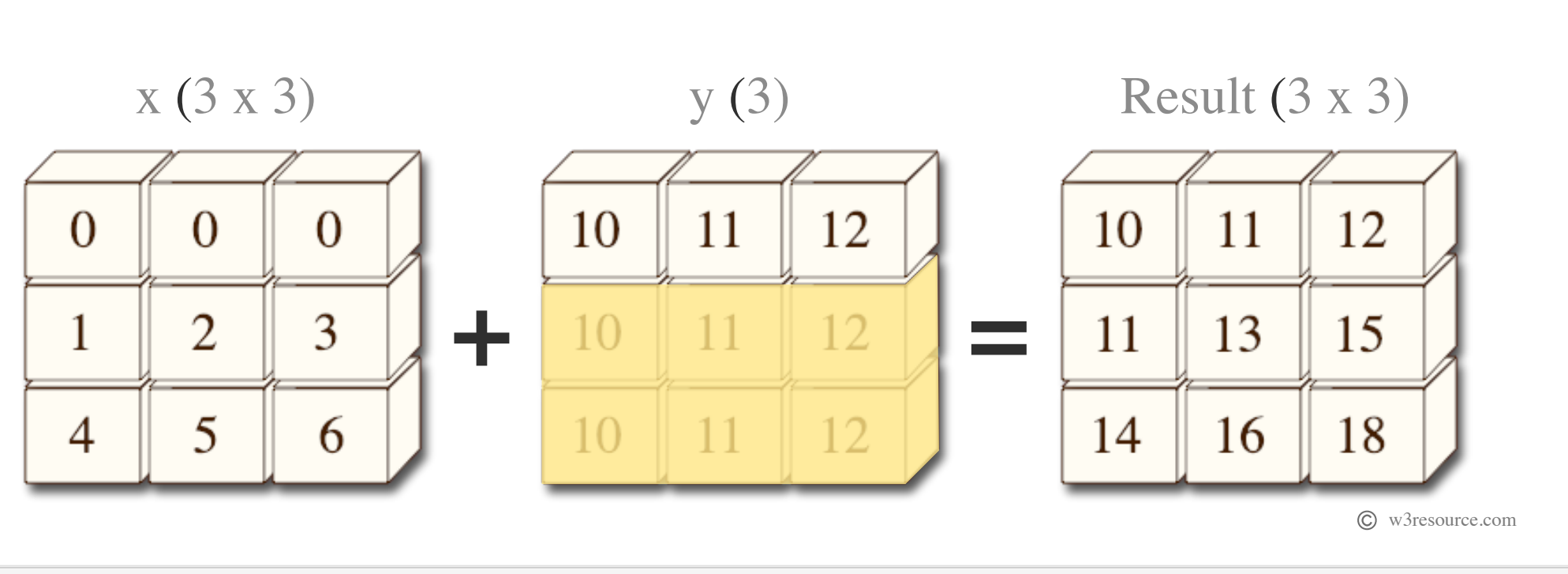
RGB Colour = 3 matrices

- RGB = three 'channels' - Red, Green and Blue. e.g. `[0, 255, 0]`



Broadcasting with `numpy.array`

- Broadcast = like 'send message to all' - apply to all elements
- Useful to apply operations to unequal sizes of array/matrix



In [6…
```
a = np.array([[1,2,3], [4,5,6], [7,8,9]])
b = np.array([10,20,30])
```

In [6…
```
a
```
Out[6…
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

In [6…
```
b
```
Out[6…
```
array([10, 20, 30])
```

In [6…
```
c = a + b
c
```
Out[6…
```
array([[11, 22, 33],
       [14, 25, 36],
       [17, 28, 39]])
```

Boolean Masking with `numpy.array`

- Boolean = True or False
- Masking = like a 'mask layer' in Photoshop, this gives you a 'view' of the structure, based on whether it fulfils the condition set (the mask)

In [6…
```
c
```
Out[6…
```
array([[11, 22, 33],
       [14, 25, 36],
       [17, 28, 39]])
```

In [6…
```
mask = (c > 20)
mask
```
Out[6…
```
array([[False,  True,  True],
       [False,  True,  True],
       [False,  True,  True]])
```

Summary - `numpy.array` vs Python `list`

`numpy.array` is useful when:

- Performing numerical computations on large datasets.
- Needing to leverage vectorized operations for performance.
- Requiring efficient memory usage.

The Python `list` is useful when:

- Dealing with heterogeneous data types.
- Performing operations that require dynamic resizing.
- Managing data that doesn't require intensive numerical computations.

This Jupyter Notebook contains exercises for you to extend your introduction to the basics with Python libraries and packages. Attempt the following exercises, which slowly build in complexity. If you get stuck, check back to the Python lecture recording on Numpy here or view the W3Schools page on Python Numpy, which includes examples, exercises and quizzes to help your understanding.

Exercises 01 - NumPy for array manipulation

This week we will work with the `numpy` library for numerical manipulations. The reference guide for numpy can be found here: https://numpy.org/doc/stable/reference/generated/numpy.ones.html. It is quite huge, but the sections of main interests for us are in the "routines" (a synonym for "functions") section, in particular: - Array creation routines - Array manipulation routines - Input and output - Statistics

Some exercises have been modified from https://www.machinelearningplus.com/python/101-numpy-exercises-python/. You can try more exercises there, if you like!

1. Import NumPy as np

In […
```
# Import the NumPy library, using `np` as an alias
```

2. Create an array of 10 zeros

In […
```
array([ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.])
```

3. Create an array of 10 ones

In […
```
array([ 1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.])
```

4. Create an array of 10 fives

In […
```
array([ 5.,  5.,  5.,  5.,  5.,  5.,  5.,  5.,  5.,  5.])
```

5. Create an array of the integers from 10 to 50

In […
```
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26,
       27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
       44, 45, 46, 47, 48, 49, 50])
```

6. Create an array of all the even integers from 10 to 50

In […
```
array([10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42,
       44, 46, 48, 50])
```

7. Create a 3x3 matrix with values ranging from 0 to 8

In […
```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

8. Create a 3x3 identity matrix

In […
```
array([[ 1.,  0.,  0.],
       [ 0.,  1.,  0.],
       [ 0.,  0.,  1.]])
```

9. Use NumPy to generate a random number between 0 and 1

In […
```
array([ 0.42829726])
```

10. Use NumPy to generate an array of 25 random numbers sampled from a standard normal distribution

In […
```
array([ 1.32031013,  1.6798602 , -0.42985892, -1.53116655,  0.85753232,
        0.87339938,  0.35668636, -1.47491157,  0.15349697,  0.99530727,
       -0.94865451, -1.69174783,  1.57525349, -0.70615234,  0.10991879,
       -0.49478947,  1.08279872,  0.76488333, -2.3039931 ,  0.35401124,
       -0.45454399, -0.64754649, -0.29391671,  0.02339861,  0.38272124])
```

11. Create the following matrix:

In […
```
array([[ 0.01,  0.02,  0.03,  0.04,  0.05,  0.06,  0.07,  0.08,  0.09,  0.1 ],
       [ 0.11,  0.12,  0.13,  0.14,  0.15,  0.16,  0.17,  0.18,  0.19,  0.2 ],
       [ 0.21,  0.22,  0.23,  0.24,  0.25,  0.26,  0.27,  0.28,  0.29,  0.3 ],
       [ 0.31,  0.32,  0.33,  0.34,  0.35,  0.36,  0.37,  0.38,  0.39,  0.4 ],
       [ 0.41,  0.42,  0.43,  0.44,  0.45,  0.46,  0.47,  0.48,  0.49,  0.5 ],
       [ 0.51,  0.52,  0.53,  0.54,  0.55,  0.56,  0.57,  0.58,  0.59,  0.6 ],
       [ 0.61,  0.62,  0.63,  0.64,  0.65,  0.66,  0.67,  0.68,  0.69,  0.7 ],
       [ 0.71,  0.72,  0.73,  0.74,  0.75,  0.76,  0.77,  0.78,  0.79,  0.8 ],
       [ 0.81,  0.82,  0.83,  0.84,  0.85,  0.86,  0.87,  0.88,  0.89,  0.9 ],
       [ 0.91,  0.92,  0.93,  0.94,  0.95,  0.96,  0.97,  0.98,  0.99,  1.  ]])
```

12. Create an array of 20 linearly spaced points between 0 and 1:

In […
```
array([ 0.        ,  0.05263158,  0.10526316,  0.15789474,  0.21052632,
        0.26315789,  0.31578947,  0.36842105,  0.42105263,  0.47368421,
        0.52631579,  0.57894737,  0.63157895,  0.68421053,  0.73684211,
        0.78947368,  0.84210526,  0.89473684,  0.94736842,  1.        ])
```

Numpy Indexing and Selection

Now you will be given a few matrices, and be asked to replicate the resulting matrix outputs:

In […
```
mat = np.arange(1,26).reshape(5,5)
mat
```
```
array([[ 1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10],
       [11, 12, 13, 14, 15],
       [16, 17, 18, 19, 20],
       [21, 22, 23, 24, 25]])
```

13. Write code to produce the following output (from referrring to mat):

```
array([[12, 13, 14, 15],
       [17, 18, 19, 20],
       [22, 23, 24, 25]])
```

In […
```
array([[12, 13, 14, 15],
       [17, 18, 19, 20],
       [22, 23, 24, 25]])
```

14. Write code below to produce the following output (from referring to mat):

```
20
```

In […
```
20
```

15. Write code below to produce the following output (from referring to mat):

```
array([[ 2],
       [ 7],
       [12]])
```

In […
```
array([[ 2],
       [ 7],
       [12]])
```

16. Write code that produces the following output (from referring to mat)

`array([21, 22, 23, 24, 25])`

In […
```
array([21, 22, 23, 24, 25])
```

17. Write code that produces the following output (from referring to mat)

```
array([[16, 17, 18, 19, 20],
       [21, 22, 23, 24, 25]])
```

In […
```
array([[16, 17, 18, 19, 20],
       [21, 22, 23, 24, 25]])
```

Now do the following

18. Get the sum of all the values in mat

In […
```
325
```

19. Get the standard deviation of the values in mat

In […
```
7.2111025509279782
```

20. Get the sum of all the columns in mat

In […
```
array([55, 60, 65, 70, 75])
```

EXTRA EXERCISES (4.5 Numpy Exercises)

1. Create a 1D array of numbers from 99 to 110

Desired output: `#> array([99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110])`

In […
```
# Write your solution here
```
```
array([ 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110])
```

2. Create a 3x3 numpy array of all False values

In […
```
# Write your solution here
```
```
array([[False, False, False],
       [False, False, False],
       [False, False, False]])
```

3. Replace all odd numbers in `arr` with -1

Sample input: `np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])`

Desired output: `#>  array([ 0, -1,  2, -1,  4, -1,  6, -1,  8, -1])`

In […
```
# Sample input
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Write your solution here:
```
```
array([ 0, -1,  2, -1,  4, -1,  6, -1,  8, -1])
```

4 Stack two arrays vertically

Consider these two numpy arrays as input:

```
a = np.arange(10).reshape(2,-1)
b = np.repeat(1, 10).reshape(2,-1)

a = [[0 1 2 3 4]
 [5 6 7 8 9]]

b = [[1 1 1 1 1]
 [1 1 1 1 1]]
```

Desired output:

```
c = [[0, 1, 2, 3, 4],
     [5, 6, 7, 8, 9],
     [1, 1, 1, 1, 1],
     [1, 1, 1, 1, 1]])
```

In […
```
a = np.arange(10).reshape(2,-1)
b = np.repeat(1, 10).reshape(2,-1)


# Write your solution here
```

Exercises of data manipulation on the "500_Person_Gender_Height_Weight_Index" dataset.

We will use a "500_Person_Gender_Height_Weight_Index" dataset containing information about height (in cm) and weights (in kg) for 500 subjects, classified by Gender. You can have a look at the dataset here in Kaggle: https://www.kaggle.com/yersever/500-person-gender-height-weight-bodymassindex You do not have to download it though. The CSV (comma separated values) file can be found in the "datasets" directory:

We are not going to use the Body Mass Index (BMI) information provided in the dataset. We will compute the BMI later on using Numpy.

Let's import the "500_Person_Gender_Height_Weight_Index" dataset. You can import the dataset keeping the text column intact, passing `dtype` ='object' as argument. In this case, though all the numeric values will be stored as bytes. We will have to set the `names` argument to `True` as the first line of the file (the "header") contains the column names.

Read more details about the textual data importing functions of Numpy here:

- `np.loadtxt()` : https://docs.scipy.org/doc/numpy-1.16.1/reference/generated/numpy.loadtxt.html#numpy.loadtxt
- `np.genfromtxt()` : https://docs.scipy.org/doc/numpy-1.16.1/reference/generated/numpy.genfromtxt.html#numpy.genfromtxt

You can see all the arguments you can supply to the functions. All the arguments that have a default value are optional.

In […
```
import os
# Import `height-weight` keeping the text column intact.
localfilepath = os.path.join('/Users/nick/Documents/GitHub/Python-Programming-for-Data-Science-Introduction/ND IPYNB SLIDES/04 Numpy/50
#localfilepath = os.path.join('500_Person_Gender_Height_Weight_Index.csv')
hw_dataset = np.genfromtxt(localfilepath, delimiter=',', names=True, dtype='object')
hw_dataset
```
```
array([(b'Male', b'174', b'96', b'4'), (b'Male', b'189', b'87', b'2'),
       (b'Female', b'185', b'110', b'4'),
       (b'Female', b'195', b'104', b'3'), (b'Male', b'149', b'61', b'3'),
       (b'Male', b'189', b'104', b'3'), (b'Male', b'147', b'92', b'5'),
       (b'Male', b'154', b'111', b'5'), (b'Male', b'174', b'90', b'3'),
       (b'Female', b'169', b'103', b'4'), (b'Male', b'195', b'81', b'2'),
       (b'Female', b'159', b'80', b'4'),
       (b'Female', b'192', b'101', b'3'), (b'Male', b'155', b'51', b'2'),
       (b'Male', b'191', b'79', b'2'), (b'Female', b'153', b'107', b'5'),
       (b'Female', b'157', b'110', b'5'), (b'Male', b'140', b'129', b'5'),
       (b'Male', b'144', b'145', b'5'), (b'Male', b'172', b'139', b'5'),
       (b'Male', b'157', b'110', b'5'), (b'Female', b'153', b'149', b'5'),
       (b'Female', b'169', b'97', b'4'), (b'Male', b'185', b'139', b'5'),
       (b'Female', b'172', b'67', b'2'), (b'Female', b'151', b'64', b'3'),
       (b'Male', b'190', b'95', b'3'), (b'Male', b'187', b'62', b'1'),
       (b'Female', b'163', b'159', b'5'), (b'Male', b'179', b'152', b'5'),
       (b'Male', b'153', b'121', b'5'), (b'Male', b'178', b'52', b'1'),
       (b'Female', b'195', b'65', b'1'),
       (b'Female', b'160', b'131', b'5'),
       (b'Female', b'157', b'153', b'5'),
       (b'Female', b'189', b'132', b'4'),
       (b'Female', b'197', b'114', b'3'), (b'Male', b'244', b'80', b'4'),
       (b'Female', b'171', b'152', b'5'),
       (b'Female', b'185', b'81', b'2'),
       (b'Female', b'175', b'120', b'4'),
       (b'Female', b'149', b'108', b'5'), (b'Male', b'157', b'56', b'2'),
       (b'Male', b'161', b'118', b'5'), (b'Female', b'182', b'126', b'4'),
       (b'Male', b'185', b'76', b'2'), (b'Female', b'188', b'122', b'4'),
       (b'Male', b'181', b'111', b'4'), (b'Male', b'161', b'72', b'3'),
       (b'Male', b'140', b'152', b'5'), (b'Female', b'168', b'135', b'5'),
       (b'Female', b'176', b'54', b'1'), (b'Male', b'163', b'110', b'5'),
       (b'Male', b'172', b'105', b'4'), (b'Male', b'196', b'116', b'4'),
       (b'Female', b'187', b'89', b'3'), (b'Male', b'172', b'92', b'4'),
       (b'Male', b'178', b'127', b'5'), (b'Female', b'164', b'70', b'3'),
       (b'Male', b'143', b'88', b'5'), (b'Female', b'191', b'54', b'0'),
       (b'Female', b'141', b'143', b'5'), (b'Male', b'193', b'54', b'0'),
       (b'Male', b'190', b'83', b'2'), (b'Male', b'175', b'135', b'5'),
       (b'Female', b'179', b'158', b'5'),
       (b'Female', b'172', b'96', b'4'), (b'Female', b'168', b'59', b'2'),
       (b'Female', b'164', b'82', b'4'),
       (b'Female', b'194', b'136', b'4'),
       (b'Female', b'153', b'51', b'2'), (b'Male', b'178', b'117', b'4'),
       (b'Male', b'141', b'80', b'5'), (b'Male', b'180', b'75', b'2'),
       (b'Female', b'185', b'100', b'3'),
       (b'Female', b'197', b'154', b'4'), (b'Male', b'165', b'104', b'4'),
       (b'Female', b'168', b'90', b'4'),
       (b'Female', b'176', b'122', b'4'), (b'Male', b'181', b'51', b'0'),
       (b'Male', b'164', b'75', b'3'), (b'Female', b'166', b'140', b'5'),
       (b'Female', b'190', b'105', b'3'), (b'Male', b'186', b'118', b'4'),
       (b'Male', b'168', b'123', b'5'), (b'Male', b'198', b'50', b'0'),
       (b'Female', b'175', b'141', b'5'), (b'Male', b'145', b'117', b'5'),
       (b'Female', b'159', b'104', b'5'),
       (b'Female', b'185', b'140', b'5'),
       (b'Female', b'178', b'154', b'5'),
       (b'Female', b'183', b'96', b'3'),
       (b'Female', b'194', b'111', b'3'), (b'Male', b'177', b'61', b'2'),
       (b'Male', b'197', b'119', b'4'), (b'Female', b'170', b'156', b'5'),
       (b'Male', b'142', b'69', b'4'), (b'Male', b'160', b'139', b'5'),
       (b'Male', b'195', b'69', b'1'), (b'Female', b'190', b'50', b'0'),
       (b'Male', b'199', b'156', b'4'), (b'Male', b'154', b'105', b'5'),
       (b'Male', b'161', b'155', b'5'), (b'Female', b'198', b'145', b'4'),
       (b'Female', b'192', b'140', b'4'), (b'Male', b'195', b'126', b'4'),
       (b'Male', b'166', b'160', b'5'), (b'Male', b'159', b'154', b'5'),
       (b'Female', b'181', b'106', b'4'), (b'Male', b'149', b'66', b'3'),
       (b'Female', b'150', b'70', b'4'),
       (b'Female', b'146', b'157', b'5'), (b'Male', b'190', b'135', b'4'),
       (b'Female', b'192', b'90', b'2'), (b'Female', b'177', b'96', b'4'),
       (b'Male', b'148', b'60', b'3'), (b'Female', b'165', b'57', b'2'),
       (b'Female', b'146', b'104', b'5'), (b'Male', b'144', b'108', b'5'),
       (b'Female', b'176', b'156', b'5'),
       (b'Female', b'168', b'87', b'4'), (b'Male', b'187', b'122', b'4'),
       (b'Male', b'187', b'138', b'4'), (b'Female', b'184', b'160', b'5'),
       (b'Female', b'158', b'149', b'5'), (b'Male', b'158', b'96', b'4'),
       (b'Male', b'194', b'115', b'4'), (b'Female', b'145', b'79', b'4'),
       (b'Male', b'182', b'151', b'5'), (b'Male', b'154', b'54', b'2'),
       (b'Female', b'168', b'139', b'5'),
       (b'Female', b'187', b'70', b'2'),
       (b'Female', b'158', b'153', b'5'),
       (b'Female', b'167', b'110', b'4'),
       (b'Female', b'171', b'155', b'5'),
       (b'Female', b'183', b'150', b'5'),
       (b'Female', b'190', b'156', b'5'), (b'Male', b'194', b'108', b'3'),
       (b'Male', b'171', b'147', b'5'), (b'Male', b'159', b'124', b'5'),
       (b'Female', b'169', b'54', b'2'), (b'Female', b'167', b'85', b'4'),
       (b'Male', b'180', b'149', b'5'), (b'Male', b'163', b'123', b'5'),
       (b'Male', b'140', b'79', b'5'), (b'Male', b'197', b'125', b'4'),
       (b'Male', b'194', b'106', b'3'), (b'Female', b'140', b'146', b'5'),
       (b'Male', b'195', b'98', b'3'), (b'Female', b'168', b'115', b'3'),
       (b'Female', b'196', b'50', b'0'), (b'Male', b'140', b'52', b'3'),
       (b'Female', b'150', b'60', b'3'),
       (b'Female', b'168', b'140', b'5'),
       (b'Female', b'155', b'111', b'5'),
       (b'Female', b'179', b'103', b'4'),
       (b'Female', b'182', b'84', b'3'), (b'Male', b'168', b'160', b'5'),
       (b'Female', b'187', b'102', b'3'), (b'Male', b'181', b'105', b'4'),
       (b'Male', b'199', b'99', b'2'), (b'Female', b'184', b'76', b'2'),
       (b'Male', b'192', b'101', b'3'), (b'Female', b'182', b'143', b'5'),
       (b'Female', b'172', b'111', b'4'), (b'Male', b'181', b'78', b'2'),
       (b'Male', b'176', b'109', b'4'), (b'Female', b'156', b'106', b'5'),
       (b'Female', b'151', b'67', b'3'), (b'Female', b'188', b'80', b'2'),
       (b'Male', b'187', b'136', b'4'), (b'Male', b'174', b'138', b'5'),
       (b'Male', b'167', b'151', b'5'), (b'Female', b'196', b'131', b'4'),
       (b'Male', b'197', b'149', b'4'), (b'Female', b'185', b'119', b'4'),
       (b'Female', b'170', b'102', b'4'),
       (b'Female', b'181', b'94', b'3'),
       (b'Female', b'166', b'126', b'5'), (b'Male', b'188', b'100', b'3'),
       (b'Female', b'162', b'74', b'3'), (b'Male', b'177', b'117', b'4'),
       (b'Male', b'162', b'97', b'4'), (b'Male', b'180', b'73', b'2'),
       (b'Female', b'192', b'108', b'3'), (b'Male', b'165', b'80', b'3'),
       (b'Female', b'167', b'135', b'5'),
       (b'Female', b'182', b'84', b'3'),
       (b'Female', b'161', b'134', b'5'), (b'Male', b'158', b'95', b'4'),
       (b'Male', b'141', b'85', b'5'), (b'Male', b'154', b'100', b'5'),
       (b'Male', b'165', b'105', b'4'), (b'Female', b'142', b'137', b'5'),
       (b'Male', b'141', b'94', b'5'), (b'Male', b'145', b'108', b'5'),
       (b'Male', b'157', b'74', b'4'), (b'Female', b'177', b'117', b'4'),
       (b'Female', b'166', b'144', b'5'), (b'Male', b'193', b'151', b'5'),
       (b'Male', b'184', b'57', b'1'), (b'Male', b'179', b'93', b'3'),
       (b'Female', b'156', b'89', b'4'), (b'Male', b'182', b'104', b'4'),
       (b'Male', b'145', b'160', b'5'), (b'Female', b'150', b'87', b'4'),
       (b'Male', b'145', b'99', b'5'), (b'Female', b'196', b'122', b'4'),
       (b'Male', b'191', b'96', b'3'), (b'Female', b'148', b'67', b'4'),
       (b'Female', b'150', b'84', b'4'), (b'Male', b'148', b'155', b'5'),
       (b'Female', b'153', b'146', b'5'),
       (b'Female', b'196', b'159', b'5'),
       (b'Female', b'185', b'52', b'0'),
       (b'Female', b'171', b'131', b'5'),
       (b'Female', b'143', b'118', b'5'),
       (b'Female', b'142', b'86', b'5'),
       (b'Female', b'141', b'126', b'5'), (b'Male', b'159', b'109', b'5'),
       (b'Female', b'173', b'82', b'2'), (b'Male', b'183', b'138', b'5'),
       (b'Female', b'152', b'90', b'4'), (b'Male', b'178', b'140', b'5'),
       (b'Male', b'188', b'54', b'0'), (b'Female', b'155', b'144', b'5'),
       (b'Male', b'166', b'70', b'3'), (b'Male', b'188', b'123', b'4'),
       (b'Female', b'171', b'120', b'5'), (b'Male', b'179', b'130', b'5'),
       (b'Female', b'186', b'137', b'4'),
       (b'Female', b'153', b'78', b'2'), (b'Female', b'184', b'86', b'3'),
       (b'Female', b'177', b'81', b'3'), (b'Male', b'145', b'78', b'4'),
       (b'Male', b'170', b'81', b'3'), (b'Male', b'181', b'141', b'5'),
       (b'Male', b'165', b'155', b'5'), (b'Female', b'174', b'65', b'2'),
       (b'Female', b'146', b'110', b'5'), (b'Male', b'178', b'85', b'3'),
       (b'Male', b'166', b'61', b'2'), (b'Male', b'191', b'62', b'1'),
       (b'Female', b'177', b'155', b'5'),
       (b'Female', b'183', b'50', b'0'), (b'Male', b'151', b'114', b'5'),
       (b'Male', b'182', b'98', b'3'), (b'Female', b'142', b'159', b'5'),
       (b'Female', b'188', b'90', b'3'), (b'Male', b'161', b'89', b'4'),
       (b'Male', b'153', b'70', b'3'), (b'Male', b'140', b'143', b'5'),
       (b'Male', b'169', b'141', b'5'), (b'Female', b'162', b'159', b'5'),
       (b'Male', b'183', b'147', b'5'), (b'Female', b'162', b'58', b'2'),
       (b'Female', b'172', b'109', b'4'),
       (b'Female', b'150', b'119', b'5'),
       (b'Female', b'169', b'145', b'5'),
       (b'Female', b'184', b'132', b'4'), (b'Male', b'159', b'104', b'5'),
       (b'Male', b'163', b'131', b'5'), (b'Male', b'156', b'137', b'5'),
       (b'Female', b'157', b'52', b'2'), (b'Male', b'147', b'84', b'4'),
       (b'Male', b'141', b'86', b'5'), (b'Male', b'173', b'139', b'5'),
       (b'Male', b'154', b'145', b'5'), (b'Male', b'168', b'148', b'5'),
       (b'Male', b'168', b'50', b'1'), (b'Male', b'145', b'130', b'5'),
       (b'Male', b'152', b'103', b'5'), (b'Female', b'187', b'121', b'4'),
       (b'Female', b'163', b'57', b'0'), (b'Male', b'178', b'83', b'3'),
       (b'Female', b'187', b'94', b'3'),
       (b'Female', b'179', b'114', b'4'), (b'Male', b'190', b'80', b'2'),
       (b'Male', b'172', b'75', b'3'), (b'Male', b'188', b'57', b'1'),
       (b'Male', b'193', b'65', b'1'), (b'Female', b'147', b'126', b'5'),
       (b'Female', b'147', b'94', b'5'), (b'Male', b'166', b'107', b'4'),
       (b'Female', b'192', b'139', b'4'), (b'Male', b'181', b'139', b'4'),
       (b'Male', b'150', b'74', b'4'), (b'Male', b'178', b'160', b'5'),
       (b'Female', b'156', b'52', b'2'), (b'Male', b'149', b'100', b'5'),
       (b'Male', b'156', b'74', b'4'), (b'Male', b'183', b'105', b'3'),
       (b'Female', b'162', b'68', b'3'), (b'Female', b'165', b'83', b'4'),
       (b'Female', b'168', b'143', b'5'), (b'Male', b'160', b'156', b'5'),
       (b'Female', b'169', b'88', b'2'), (b'Female', b'140', b'76', b'4'),
       (b'Female', b'187', b'92', b'3'), (b'Male', b'151', b'82', b'4'),
       (b'Female', b'186', b'140', b'5'), (b'Male', b'182', b'108', b'4'),
       (b'Male', b'188', b'81', b'2'), (b'Male', b'179', b'110', b'4'),
       (b'Female', b'156', b'126', b'5'), (b'Male', b'188', b'114', b'4'),
       (b'Male', b'183', b'153', b'5'), (b'Male', b'144', b'88', b'5'),
       (b'Male', b'196', b'69', b'1'), (b'Male', b'171', b'141', b'5'),
       (b'Male', b'171', b'147', b'5'), (b'Female', b'180', b'156', b'5'),
       (b'Male', b'191', b'146', b'5'), (b'Female', b'179', b'67', b'2'),
       (b'Female', b'180', b'60', b'2'),
       (b'Female', b'154', b'132', b'5'), (b'Male', b'188', b'99', b'3'),
       (b'Male', b'142', b'135', b'5'), (b'Male', b'170', b'95', b'4'),
       (b'Male', b'152', b'141', b'5'), (b'Female', b'190', b'118', b'4'),
       (b'Female', b'181', b'111', b'4'), (b'Male', b'153', b'104', b'5'),
       (b'Male', b'187', b'140', b'5'), (b'Female', b'144', b'66', b'4'),
       (b'Female', b'148', b'54', b'2'), (b'Female', b'199', b'92', b'2'),
       (b'Female', b'167', b'85', b'4'), (b'Female', b'164', b'71', b'3'),
       (b'Female', b'185', b'102', b'3'),
       (b'Female', b'164', b'160', b'5'), (b'Male', b'142', b'71', b'4'),
       (b'Male', b'165', b'68', b'2'), (b'Female', b'172', b'62', b'2'),
       (b'Female', b'157', b'56', b'2'), (b'Male', b'155', b'57', b'2'),
       (b'Female', b'167', b'153', b'5'),
       (b'Female', b'164', b'126', b'5'),
       (b'Female', b'189', b'125', b'4'),
       (b'Female', b'161', b'145', b'5'),
       (b'Female', b'155', b'71', b'3'),
       (b'Female', b'171', b'118', b'4'),
       (b'Female', b'154', b'92', b'4'), (b'Male', b'179', b'83', b'3'),
       (b'Male', b'170', b'115', b'4'), (b'Female', b'184', b'106', b'4'),
       (b'Female', b'191', b'68', b'2'), (b'Male', b'162', b'58', b'2'),
       (b'Male', b'178', b'138', b'5'), (b'Female', b'157', b'60', b'2'),
       (b'Male', b'184', b'83', b'2'), (b'Male', b'197', b'88', b'2'),
       (b'Female', b'160', b'51', b'2'), (b'Male', b'184', b'153', b'5'),
       (b'Male', b'190', b'50', b'0'), (b'Male', b'174', b'90', b'3'),
       (b'Female', b'189', b'124', b'4'),
       (b'Female', b'186', b'143', b'5'),
       (b'Female', b'180', b'58', b'1'),
       (b'Female', b'186', b'148', b'4'),
       (b'Female', b'193', b'61', b'1'), (b'Male', b'161', b'103', b'4'),
       (b'Female', b'151', b'158', b'5'),
       (b'Female', b'195', b'147', b'4'),
       (b'Female', b'184', b'152', b'5'), (b'Male', b'141', b'80', b'5'),
       (b'Female', b'185', b'94', b'3'),
       (b'Female', b'186', b'127', b'4'), (b'Male', b'142', b'131', b'5'),
       (b'Female', b'147', b'67', b'4'), (b'Male', b'151', b'62', b'3'),
       (b'Female', b'160', b'124', b'5'), (b'Male', b'185', b'60', b'1'),
       (b'Female', b'163', b'63', b'2'), (b'Male', b'174', b'95', b'4'),
       (b'Female', b'150', b'144', b'5'), (b'Male', b'142', b'91', b'5'),
       (b'Male', b'178', b'142', b'5'), (b'Female', b'154', b'96', b'5'),
       (b'Male', b'176', b'87', b'3'), (b'Male', b'159', b'120', b'5'),
       (b'Male', b'191', b'62', b'1'), (b'Male', b'177', b'117', b'4'),
       (b'Male', b'151', b'154', b'5'), (b'Female', b'182', b'149', b'5'),
       (b'Female', b'197', b'72', b'2'), (b'Male', b'146', b'138', b'5'),
       (b'Female', b'160', b'83', b'4'), (b'Female', b'157', b'66', b'3'),
       (b'Female', b'150', b'50', b'2'), (b'Female', b'167', b'58', b'2'),
       (b'Female', b'180', b'70', b'2'), (b'Female', b'183', b'76', b'2'),
       (b'Female', b'183', b'87', b'3'),
       (b'Female', b'152', b'154', b'5'),
       (b'Female', b'164', b'71', b'3'), (b'Male', b'187', b'96', b'3'),
       (b'Male', b'169', b'136', b'5'), (b'Female', b'149', b'61', b'3'),
       (b'Male', b'163', b'137', b'5'), (b'Female', b'195', b'104', b'3'),
       (b'Male', b'174', b'107', b'4'), (b'Male', b'182', b'70', b'2'),
       (b'Male', b'169', b'110', b'4'), (b'Male', b'193', b'130', b'4'),
       (b'Male', b'148', b'141', b'5'), (b'Male', b'186', b'68', b'2'),
       (b'Male', b'165', b'143', b'5'), (b'Female', b'146', b'123', b'5'),
       (b'Female', b'166', b'133', b'5'), (b'Male', b'179', b'56', b'1'),
       (b'Female', b'177', b'101', b'4'), (b'Male', b'181', b'154', b'5'),
       (b'Female', b'161', b'154', b'5'),
       (b'Female', b'157', b'103', b'5'),
       (b'Female', b'169', b'98', b'4'),
       (b'Female', b'152', b'114', b'5'),
       (b'Female', b'162', b'64', b'2'), (b'Male', b'162', b'130', b'5'),
       (b'Female', b'177', b'61', b'2'), (b'Female', b'195', b'61', b'1'),
       (b'Male', b'140', b'146', b'5'), (b'Female', b'186', b'146', b'5'),
       (b'Female', b'178', b'107', b'4'), (b'Male', b'174', b'54', b'1'),
       (b'Female', b'180', b'59', b'1'), (b'Male', b'188', b'141', b'4'),
       (b'Female', b'187', b'130', b'4'),
       (b'Female', b'153', b'77', b'4'), (b'Female', b'165', b'95', b'4'),
       (b'Female', b'178', b'79', b'2'),
       (b'Female', b'163', b'154', b'5'),
       (b'Female', b'150', b'97', b'5'), (b'Male', b'179', b'127', b'4'),
       (b'Male', b'165', b'62', b'2'), (b'Male', b'168', b'158', b'5'),
       (b'Female', b'153', b'133', b'5'), (b'Male', b'184', b'157', b'5'),
       (b'Male', b'188', b'65', b'1'), (b'Female', b'166', b'153', b'5'),
       (b'Female', b'172', b'116', b'4'), (b'Male', b'182', b'73', b'2'),
       (b'Male', b'143', b'149', b'5'), (b'Male', b'152', b'146', b'5'),
       (b'Female', b'186', b'128', b'4'), (b'Male', b'159', b'140', b'5'),
       (b'Male', b'146', b'70', b'4'), (b'Female', b'176', b'121', b'4'),
       (b'Female', b'146', b'101', b'5'), (b'Male', b'159', b'145', b'5'),
       (b'Male', b'162', b'157', b'5'), (b'Female', b'172', b'90', b'4'),
       (b'Female', b'169', b'121', b'5'), (b'Male', b'182', b'50', b'0'),
       (b'Female', b'183', b'79', b'2'), (b'Male', b'176', b'77', b'2'),
       (b'Female', b'188', b'128', b'4'),
       (b'Female', b'175', b'83', b'2'), (b'Male', b'154', b'81', b'4'),
       (b'Female', b'184', b'147', b'5'), (b'Male', b'179', b'123', b'4'),
       (b'Male', b'152', b'132', b'5'), (b'Male', b'179', b'56', b'1'),
       (b'Female', b'145', b'141', b'5'),
       (b'Female', b'181', b'80', b'2'), (b'Male', b'158', b'127', b'5'),
       (b'Female', b'188', b'99', b'3'), (b'Male', b'145', b'142', b'5'),
       (b'Male', b'161', b'115', b'5'), (b'Male', b'198', b'109', b'3'),
       (b'Male', b'147', b'142', b'5'), (b'Male', b'154', b'112', b'5'),
       (b'Female', b'178', b'65', b'2'), (b'Male', b'195', b'153', b'5'),
       (b'Female', b'167', b'79', b'3'), (b'Male', b'183', b'131', b'4'),
       (b'Female', b'164', b'142', b'5'), (b'Male', b'167', b'64', b'2'),
       (b'Female', b'151', b'55', b'2'),
       (b'Female', b'147', b'107', b'5'),
       (b'Female', b'155', b'115', b'5'),
       (b'Female', b'172', b'108', b'4'),
       (b'Female', b'142', b'86', b'5'), (b'Male', b'146', b'85', b'4'),
       (b'Female', b'188', b'115', b'4'), (b'Male', b'173', b'111', b'4'),
       (b'Female', b'160', b'109', b'5'), (b'Male', b'187', b'80', b'2'),
       (b'Male', b'198', b'136', b'4'), (b'Female', b'179', b'150', b'5'),
       (b'Female', b'164', b'59', b'2'),
       (b'Female', b'146', b'147', b'5'),
       (b'Female', b'198', b'50', b'0'), (b'Female', b'170', b'53', b'1'),
       (b'Male', b'152', b'98', b'5'), (b'Female', b'150', b'153', b'5'),
       (b'Female', b'184', b'121', b'4'),
       (b'Female', b'141', b'136', b'5'), (b'Male', b'150', b'95', b'5'),
       (b'Male', b'173', b'131', b'5')],
      dtype=[('Gender', 'O'), ('Height', 'O'), ('Weight', 'O'), ('Index', 'O')])
```

Otherwise you can import the "500_Person_Gender_Height_Weight_Index" with the correct datatypes for the numeric values, specifying `dtype` for each column.

In […

```python
localfilepath = os.path.join('..', 'datasets', '500_Person_Gender_Height_Weight_Index.csv')
hw_dataset = np.genfromtxt(
    localfilepath, delimiter=',', names=True, dtype=[ '|S15', np.float64, np.float64, np.int32]
)
hw_dataset[:4]
```

```
array([(b'Male', 174.,  96., 4), (b'Male', 189.,  87., 2),
       (b'Female', 185., 110., 4), (b'Female', 195., 104., 3)],
      dtype=[('Gender', 'S15'), ('Height', '<f8'), ('Weight', '<f8'), ('Index', '<i4')])
```

5. How to convert a 1d array of tuples to a 2d numpy array

Convert the `hw_dataset` to a numeric-only 2D array `hw_data` by omitting the "Gender" text column and the "Index" numeric field. Create a `hw_label` 1D array containing only the "Gender" text field. Keep the same indexing/order as in the original array.

In […

```python
# Write your solution here
```

6. Split the datasets in two groups

Split the dataset in `hw_data` according to the labels in `hw_labels`. Hint: You can create a dictionary with the two different labels ("Male" and "Female") as keys, and the two split datasets as values

In […

```python
# Write your solution here
```

7. Compute the monovariate statistics for each group

For each label compute the key statistics for both weight and height.

- mean
- median
- standard deviation
- variance
- interquartile range

Read carefully the numpy documentation for the statistics routines. Answer to the following questions:

1. Which is the gender with the highest mean value for `height`?
2. Which is the gender with the smallest median value for `weight`?
3. Which is the gender with the shortest interquartile range for `weight`?

In […

```python
# Write your solution here
```

8. Compute the covarance and correlation matrix for each group

1. Compute the `height` - `weight` covariance matrix for each gender. Are the values on the diagonal matching the values computed with the variance functions in the previous step? If not, can you understand why, and how you can obtain coherent values?
2. Compute the `height` - `weight` correlation matrix for each gender.

In […

```python
# Compute the covariance matrix here:
```

In […

```python
# Compute the correlation matrix here
```

9. How to create a new column from existing columns of a numpy array

Create a new column for "Mass-body Index" ("MBI") in `hw_data`, where MBI is:

$$MBI = \frac{weight[kg]}{(height[m])^2}$$

In […

```python
# Compute MBI here:
```

10. Convert a quantitative variable to a categorical one

Now create an array `hw_mbi_labels` where you assign each record in `hw_data` to one of these categories: - 'UNDERWEIGHT': MBI < 18.5 - 'NORMAL': 18.5 =< MBI < 25 - 'OVERWEIGHT': 25 =< MBI < 30 - 'OBESE': MBI >= 30

Then, count the number of occurrences of each category per gender.

In […

```python
# Label the record here
# Hint: write a function to assign labels to the record then use the numpy appropriate function
# to apply the function to each row
```

11. Compute the frequencies of each category within genders

Compute the percentages of the four categories ('UNDERWEIGHT', 'NORMAL', 'OVERWEIGHT', 'OBESE'), and anwer these questions:

- Which gender has the highest percentage of 'OBESE' subjects
- Which gender has the highest petcentahe of 'subjects that are neither 'OVERWEIGHT' nor 'OBESE'?

In […

```python
# Compute the percentages here:
```

12. Normalize an array so the values range exactly between 0 and 1

Normalization is an important pre-processing step before feeding a dataset to a data science (e.g a machine learning) algorithm. Create a normalized form of `hw_data`'s "height" and "weight" whose values range exactly between 0 and 1 so that the minimum has value 0 and maximum has value 1.

In […

```python
# Write your solution here
```

This website does not host notebooks, it only renders notebooks available on other websites.

Delivered by Fastly, Rendered by OVHcloud

nbviewer GitHub repository.

nbconvert version: 7.16.6

Rendered a few seconds ago